



Módulo 8 Actividad Final

Reto

MLOps y Despliegue de Modelos

Título:

"Pipeline MLOps para un Sistema de Predicción de Churn con Despliegue en API y Seguimiento con MLflow"

Contexto del reto (realista para empresa)

Una empresa de telecomunicaciones quiere automatizar y estandarizar todo el ciclo de vida de un modelo de IA que predice la fuga de clientes (*churn*). Actualmente:

- Cada científico de datos trabaja en notebooks aislados
- No existe auditoría ni trazabilidad de experimentos
- El despliegue está hecho manualmente
- No hay monitorización del rendimiento

El área desea implementar un **pipeline MLOps completo**, modular y reproducible.

El reto del estudiante será implementarlo **desde cero**.

Objetivos de aprendizaje del reto

Al finalizar, el estudiante será capaz de:

1. Diseñar un **pipeline MLOps modular** según normas como:
 - ISO/IEC 25010
 - CRISP-DM
 - MLOps v2.0
 - Buenas prácticas de versionado y reproducibilidad
2. Implementar:
 - Ingesta + validación
 - Preprocesamiento y split
 - Entrenamiento con MLflow
 - Evaluación automatizada
 - API REST con FastAPI
 - Despliegue en Docker
 - Monitorización y reentrenamiento automático
3. Registrar experimentos, métricas, parámetros y artefactos.
4. Desplegar su modelo funcionalmente a través de un endpoint /predict.



Dataset para el reto

Usarán el dataset trabajado en clase:

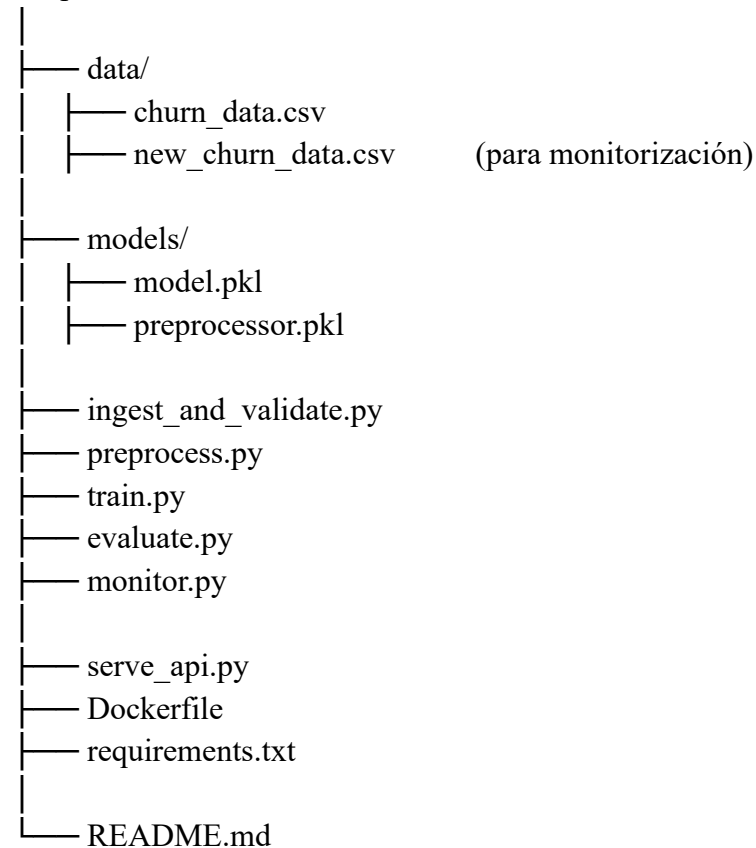
/data/churn_data.csv

Columnas:

- age
- tenure
- monthly_fee
- num_products
- has_partner
- has_dependents
- churn

Estructura mínima del proyecto (requerida)

mlops-churn/





REQUISITOS DEL RETO

1 Fase de Ingesta y Validación

Implementar:

- ✓ Validación de esquema
- ✓ Validación de porcentaje de nulos
- ✓ Registro en MLflow de la validación (opcional, bonus)

2 Fase de Preprocesamiento

- ✓ Selección de features
- ✓ Escalado con StandardScaler
- ✓ División train/test
- ✓ Guardado del preprocesador y datasets versionados

3 Entrenamiento con MLflow

- ✓ Entrenar un modelo de clasificación
- ✓ Loggear:
 - hiperparámetros
 - métricas
 - artefactos
- ✓ Registrar modelo con nombre "churn_model"

4 Evaluación automática

- ✓ Ejecutar métricas: Accuracy, F1, ROC-AUC
- ✓ Registrar métricas en MLflow
- ✓ Decidir si el modelo es "Aprobado" o "Rechazado" según umbral

5 API REST con FastAPI

Debe contener:

Endpoint 1: /predict

Entrada (JSON):

```
{  
  "age": 25,  
  "tenure": 12,  
  "monthly_fee": 50,  
  "num_products": 2,  
  "has_partner": 1,  
  "has_dependents": 0
```



```
}
```

Salida:

```
{  
  "prediction": 0,  
  "probability": 0.13  
}
```

Endpoint 2: /health

Para verificar disponibilidad.

6 Despliegue con Docker

Obligatorio:

- ✓ Imagen
- ✓ Contenedor
- ✓ API funcionando en <http://localhost:8000/docs>

7 Monitorización y Reentrenamiento

Se debe implementar:

- ✓ detección automática de bajada de rendimiento
- ✓ reentrenamiento con datos nuevos
- ✓ registro del modelo reentrenado en MLflow

Entregables

📌 1. Repositorio GitHub con:

- Código completo
- Archivo README.md con:
 - Explicación del pipeline
 - Pasos de ejecución
 - Decisiones técnicas
 - Capturas de MLflow
 - Capturas de FastAPI
 - Resultados del reentrenamiento

📌 2. Documento PDF (máx. 4 páginas):

- Arquitectura
- Diagrama del pipeline (requerido)
- Riesgos MLOps



- Controles aplicados

Rúbrica de evaluación (100 puntos)

Categoría	Puntos
Estructura del proyecto	10
Ingesta + validación	10
Preprocesamiento y versionado	15
Entrenamiento con MLflow	15
API funcional con FastAPI	15
Dockerización	10
Monitorización + reentrenamiento	15
Documentación profesional	10